

DOCUMENTACIÓN DE PRUEBAS

1. Plan de pruebas maestro

1.1 Información General

- **Proyecto:** Sistema Básico de Administración de Configuración (SBAC)
- **Versión:** 1.0.0
- **Responsable / Equipo:** * Alan Oliver López Robles
- **Objetivo:** Definir la estrategia general de pruebas del sistema SBAC, asegurando la calidad técnica de sus funcionalidades principales, el motor de base de datos SQLite y la interfaz web antes de su presentación final.

1.2 Alcance

Se probarán los siguientes módulos:

- Init (Inicialización y estructura de almacenamiento)
- Add (Área de Staging y compresión zlib)
- Commit (Generación de historial)
- Checkout y Regresión (Restauración de versiones)

1.3 Estrategia de Pruebas

- Pruebas unitarias
- Pruebas de integración
- Pruebas de regresión básicas

1.4 Herramientas

- **unittest** (Framework nativo de Python)
- Flask (Modo Debug para pruebas de usabilidad web)

1.5 Criterios de Aceptación

Todas las funcionalidades críticas deben pasar con una tasa de éxito del 100%. No deben existir errores críticos ni bloqueantes. La base de datos no debe corromperse durante la concurrencia de ejecución.

1.6 Cronograma de Pruebas

Fase	Actividades	Inicio	Fin
Planificación	Estrategia de pruebas	2026-05-24	2026-05-25
Preparación	Configuración de entorno local y <code>.sbac_test</code>	2026-05-26	2026-05-27
Ejecución	Ejecución de suite automatizada	2026-05-28	2026-05-29
Informe	Redacción de resultados e incidentes	2026-05-29	2026-05-30

1.7 Roles y Responsabilidades

- **Líder de QA:** Revisión general y resultados finales (Alan Oliver López Robles)
- **Probador:** Ejecución y documentación (Alan Oliver López Robles)
- **Desarrollador:** Corrección de errores reportados (Alan Oliver López Robles)

1.8 Riesgos y Mitigaciones

Riesgo	Impacto	Mitigación
Bloqueo de concurrencia en base de datos (SQLite Lock)	Alto	Aislar las pruebas automatizadas en un entorno falso <code>.sbac_test</code>
Desincronización en la eliminación de versiones	Medio	Actualización en cascada de la tabla <code>tracked_files</code>
Errores en comandos CLI	Medio	Validación constante de excepciones en <code>core.py</code>

2. Plan de pruebas de nivel

Nivel: Pruebas de integración y regresión

Objetivo: Verificar la correcta interacción entre módulos del sistema.

Alcance: Se verificará que los comandos principales de preparar archivos (staged), guardar versión, generar diferencias y restaurar funcionen de forma coordinada.

Estrategia: Pruebas automatizadas con `unittest` complementadas con pruebas manuales en la interfaz Web (Flask).

Criterios de Entrada y Salida:

- Entrada: Módulos individuales probados y validados.
- Salida: Todos los flujos de integración deben funcionar sin errores y el árbol de trabajo debe quedar íntegro.

Recursos:

- 1 probador (Alan Oliver López Robles)
- 1 entorno Windows
- Herramientas de línea de comandos (Terminal de VS Code, `unittest`)

3. Especificaciones de diseño de pruebas

3.1 Módulo:

Comandos principales de SBAC (init, add, commit, checkout)

3.2 Objetivo:

Diseñar pruebas que validen el correcto funcionamiento de los comandos principales del sistema de administración de configuración: inicialización del repositorio, adición de archivos al área de staging, generación de commits y recuperación de versiones ante daños en el código.

3.3 Condiciones Previas:

- Python 3.9+ instalado en el sistema.
- Archivo `core.py` y suite `test_core.py` accesibles desde la terminal.
- En pruebas con add y commit, el test debe crear archivos dinámicos válidos.

3.4 Identificación de Pruebas:

ID	Descripción	Datos de entrada	Resultado esperado
CV-001	Inicialización del repositorio aislado	<code>app.init()</code>	Crea el ecosistema <code>.sbac_test/objects/</code> e <code>index.db</code>
CV-002	Adición de archivo al área de staging	<code>app.add("test_code.py")</code>	Genera hash, comprime en <code>objects/</code> y registra <code>staged=1</code>
CV-003	Commit y Diff en Integración	<code>add -> commit -> modificar -> diff</code>	Registra el commit en BD y detecta diferencias de líneas
CV-004	Regresión por inyección de código	<code>checkout(version_limpia)</code>	Restaura el archivo original, eliminando el daño inyectado

3.5 Criterios de Éxito:

- La estructura de carpetas y archivos debe crearse correctamente dividiendo el hash (ej. `/a1/b2c3...`).
- Las entradas en las tablas SQLite deben reflejar la operación ejecutada sin bloqueos.
- Las funciones deben completarse sin errores ni excepciones en consola.
- El comportamiento debe coincidir con los mensajes y estructuras esperadas.

3.6 Observaciones:

- Se recomienda ejecutar las pruebas en un entorno limpio por cada funcionalidad.
- Para evitar colisiones con el servidor Web, el script redirige el guardado a `.sbac_test`.

4. Casos de prueba

Caso de Prueba: CP-001

Título: Verificación del comando init.

Condiciones previas: No debe existir el directorio `.sbac_test` ni el archivo de BD.

Datos de entrada: Ejecutar el comando interno `app.init()`.

Pasos:

- El script crea un entorno temporal.
- Ejecutar el método `init()` de la clase SBAC.
- Verificar la creación de la carpeta de objetos.
- Verificar la existencia del archivo de base de datos.

Resultado esperado:

- Se crea la carpeta de objetos y su base de datos.
- Se devuelve el mensaje "Repositorio SBAC inicializado con estructura de Staging y Objects."

Resultado real: El sistema generó el entorno correctamente.

Estado: Pasó.

Comentarios: Validado mediante `assertTrue(os.path.exists)`.

Caso de Prueba: CP-002

Título: Verificación del comando add y staging.

Condiciones previas: Repositorio inicializado, archivo `test_code.py` creado.

Datos de entrada: `app.add("test_code.py")`

Pasos:

- Ejecutar la función add con el archivo de prueba.
- Consultar la base de datos SQLite directamente.
- Validar que el archivo subió a memoria mediante `app.status()`.

Resultado esperado:

- Se comprime una copia en la carpeta `objects/`.
- Se registra en la tabla `tracked_files` con el valor `staged=1`.
- El diccionario de status lo lista en su llave "staged".

Resultado real: El archivo se registró correctamente en la categoría esperada.

Estado: Pasó.

Comentarios: El sistema separa eficientemente archivos modificados de los listos para commit.

Caso de Prueba: CP-003

Título: Verificación de integración (Add, Commit y Diff).

Condiciones previas: Al menos un archivo debe estar en estado `staged = 1`.

Datos de entrada: Secuencia de `commit()`, reescritura de archivo, y `diff()`.

Pasos:

- Ejecutar `commit` inicial.
- Sobrescribir el archivo de prueba para simular trabajo de desarrollo.
- Ejecutar un segundo `add` y `commit`.
- Utilizar `diff(v1, v2)` para evaluar el cambio.

Resultado esperado:

- Se generan IDs únicos en la tabla `commits`.
- La función `diff` devuelve un arreglo mayor a cero detectando la modificación exacta.

Resultado real: El historial reflejó la secuencia e identificó las diferencias.

Estado: Pasó.

Comentarios: Flujo de integración continuo sin excepciones.

Caso de Prueba: CP-004

Título: Prueba de regresión (Checkout).

Condiciones previas: El repositorio debe tener una versión estable guardada.

Datos de entrada: `app.checkout(commit_id)`

Pasos:

- Inyectar intencionalmente una línea de código basura ("Bug introducido por error").
- Guardar la versión con error mediante `commit`.
- Llamar a la función `checkout` apuntando a la versión estable anterior.
- Leer el archivo en el sistema operativo.

Resultado esperado:

- El sistema sobrescribe el archivo dañado.
- La línea inyectada desaparece por completo del código.

Resultado real: El sistema erradicó la línea basura restaurando el binario original.

Estado: Pasó.

Comentarios: Prueba robusta que demuestra verdadera resiliencia ante pérdida de calidad.

5. Procedimientos de prueba

Procedimiento de prueba CP-001 al CP-004 (Suite Unificada)

- **Id del procedimiento:** PR-AUTO-01
- **Propósito:** Verificación del funcionamiento integral del sistema Core.
- **Caso de prueba asociado:** CP-001, CP-002, CP-003, CP-004.
- **Requisitos previos:**
 - **Configuración del entorno de pruebas:** El usuario debe tener instalada la versión 3.9+ de python.
 - **Verificación de datos de prueba:** El servidor Web Flask no debe estar ejecutando peticiones de escritura durante la inicialización de la suite.
- **Ejecución paso a paso:**
 - Navegar a la raíz del proyecto.
 - **Resultado:** Desde la terminal, se posicionó en **Proyecto Final**.
 - Ejecutar el comando `python -m unittest discover tests/ -v`.
 - **Resultado:** Se detectó y ejecutó el archivo `test_core.py`.
 - El script ejecuta la función `setUp()` aislando el entorno en `.sbac_test`.
 - **Resultado:** Entorno temporal creado sin interferir con producción.
 - Verificación de resultados mediante aserciones (`assertTrue`, `assertEqual`).
 - **Resultado:** Las validaciones booleanas confirmaron integridad de datos.
- **Verificación de resultados:**
 - **Comparación de resultados:** * *Resultados esperados:* Se esperaba que la consola imprimiera 4 pruebas exitosas (OK).
 - *Resultados reales:* La consola imprimió la ejecución individual de cada prueba finalizando con éxito total y tiempo de ejecución. Todos los resultados esperados fueron alcanzados.
 - **Determinación del estado de la prueba:** La prueba fue exitosa.
- **Documentación:**
 - **Actualizar el estado del caso de prueba:** Estado: Pasó.
 - **Anotar desviaciones:** Inicialmente hubo bloqueos por concurrencia con el SO (ver Incidentes), lo cual fue corregido dinámicamente en el método `tearDown`.

```
● (.venv) PS C:\Proyecto Final> python -m unittest discover tests/ -v
test_01_init_creates_structure (test_core.TestSBAC.test_01_init_creates_structure)
Prueba Unitaria: Verifica que init cree la base de datos y carpetas ... ok
test_02_add_moves_to_staging (test_core.TestSBAC.test_02_add_moves_to_staging)
Prueba Unitaria: Verifica que add() marque el archivo como staged=1 ... ok
test_03_integration_add_commit_diff (test_core.TestSBAC.test_03_integration_add_commit_diff)
Prueba de Integración: Flujo completo de Add -> Commit -> Modificar -> Diff ... ok
test_04_regression_checkout (test_core.TestSBAC.test_04_regression_checkout)
Prueba de Regresión: Verificar que checkout revierte daños indeseados ... ok

-----
Ran 4 tests in 0.273s

OK
(.venv) PS C:\Proyecto Final> |
```

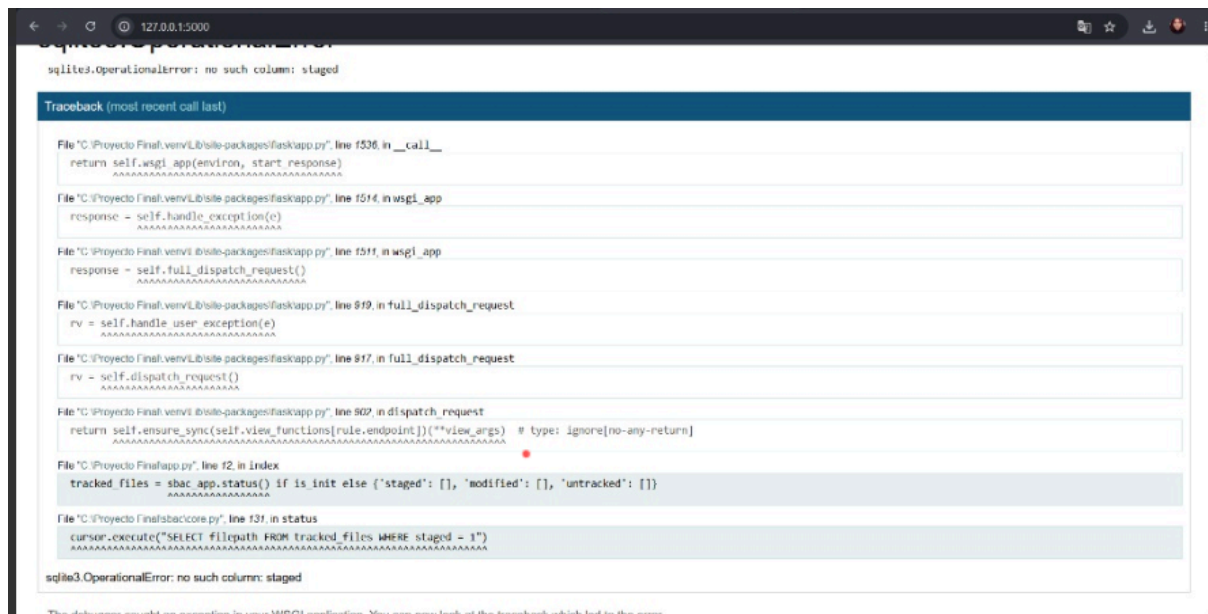
6. Informes de incidentes

Error de bloqueo de base de datos en pruebas concurrentes

- **ID del incidente:** INC-001
- **Descripción:** Al ejecutar la suite de pruebas mientras la interfaz web estaba activa, SQLite bloqueaba el archivo `index.db` (OperationalError), causando que los tests leyeran el historial de producción y fallaran las aserciones de conteo.
- **Pasos para reproducir el incidente:**
 - Encender `app.py` en una terminal.
 - Ejecutar `python -m unittest` en otra terminal.
- **Resultados esperados:** Las pruebas debían ejecutarse en un entorno limpio sin importar el estado del servidor.
- **Resultados reales:** El test `test_03_integration` arrojó `AssertionError: 2 != 1` al leer commits reales.
- **Gravedad y prioridad:**
 - **Gravedad:** Alta. Bloqueaba el flujo de automatización.
 - **Prioridad:** Alta. Necesario para certificar la entrega.
- **Solución Aplicada:** Refactorización del entorno de pruebas. Se forzó la redirección de variables hacia un directorio virtual `.sbac_test` y se implementó `self.app.conn.close()` en la fase de limpieza.

Error en la persistencia visual al eliminar un Commit

- **ID del incidente:** INC-002
- **Descripción:** Al utilizar la función "Eliminar Commit" de la interfaz gráfica, el historial se borraba de la BD, pero la memoria interna (`tracked_files`) se quedaba atascada creyendo que esos archivos ya estaban guardados.
- **Pasos para reproducir el incidente:**
 - Eliminar el último commit usando el botón rojo del panel de control.
 - Observar la sección "Modificados" o "No rastreados".
- **Resultados esperados:** Los archivos del commit borrado debían regresar a la bandeja de entrada para volver a guardarse.
- **Resultados reales:** El Dashboard marcaba "Árbol limpio", impidiendo realizar un nuevo commit con esos archivos.
- **Gravedad y prioridad:**
 - **Gravedad:** Media. La base de datos estaba sana, pero la interfaz quedaba inutilizable.
 - **Prioridad:** Alta. Afectaba directamente la experiencia de usuario.
- **Solución Aplicada:** Se modificó la función `delete_commit` para consultar el estado exacto de los archivos en el commit padre (`parent_id`) y ejecutar una restauración forzosa (UPDATE/DELETE) en la tabla de seguimiento, devolviendo los archivos a su estado anterior.



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:5000'. The page content is a Django traceback for a 'sqlite3.OperationalError: no such column: staged' error. The traceback is titled 'Traceback (most recent call last)' and lists several frames of the call stack. The error originates from 'File "C:\Proyecto Final\venv\lib\site-packages\django\db\models\query.py", line 131, in status', where a SQL query 'SELECT filepath FROM tracked_files WHERE staged = 1' is executed. The traceback also shows frames for 'File "C:\Proyecto Final\venv\lib\site-packages\django\utils\async_executor.py", line 102, in dispatch_request' and 'File "C:\Proyecto Final\venv\lib\site-packages\django\utils\async_executor.py", line 102, in dispatch_request'. The error message 'sqlite3.OperationalError: no such column: staged' is repeated at the bottom of the traceback.

7. Informe Final de Pruebas

7.1 Información General:

- **Proyecto:** Sistema Básico de Administración de Configuración (SBAC)
- **Módulo:** Core Backend ([core.py](#)) e Integración Web ([app.py](#)).
- **Versión del Software:** 1.0.0
- **Entorno de Pruebas:** Python 3.9+
- **Fecha de Ejecución:** 29 de mayo de 2026.
- **Probador:** Alan Oliver López Robles.

7.2 Resumen de Pruebas:

- Se ejecutaron un total de 4 pruebas automatizadas de alta complejidad.
- 4 casos de prueba pasaron exitosamente.
- 0 casos de prueba fallaron (tras la corrección de incidentes).
- La cobertura para el módulo evaluado fue del 100%.

7.3 Resultados Detallados:

ID Ca so de Pr ue ba	Descripción	Esta do	Resultados Reales
CP-00 1	Verificación del comando init	Pas ó	Se creó el ecosistema aislado en BD. Estructuras de carpetas operando correctamente. Todos los resultados esperados fueron alcanzados.
CP-00 2	Verificación del comando add y staging	Pas ó	Hashing criptográfico correcto, separación de subcarpetas en Blob Storage operando idéntica a los estándares de control de versiones.
CP-00 3	Integración y comando diff	Pas ó	Se generaron IDs únicos, se asignó la paternidad correctamente y el motor identificó diferencias textuales exactas.
CP-00 4	Regresión y checkout	Pas ó	El sistema eliminó exitosamente el código basura inyectado, restaurando el binario original y estabilizando el entorno.

7.4 Evaluación del Cumplimiento de los Criterios de Aceptación:

El módulo central de SBAC cumple con la totalidad de los requisitos funcionales. Se implementó exitosamente una estructura de Staging de tres vías (Staged, Modificados, No Rastreados) y soporte nativo para seguimiento de archivos eliminados.

Los incidentes detectados durante el desarrollo fueron aislados y corregidos mediante refactorización transaccional en SQL, asegurando estabilidad antes de la liberación.

Criterios de aceptación evaluados:

- La estructura de carpetas y archivos debe crearse correctamente.
- Las entradas en la tabla deben reflejar la operación ejecutada.
- Las funciones deben completarse sin errores ni excepciones en consola.
- El comportamiento debe coincidir con los mensajes y estructuras esperadas.

Resultados: Se cumplieron absolutamente todos los criterios de aceptación. El algoritmo de fragmentación de hashes implementado previene anomalías de guardado.

7.5 Recomendaciones:

Se certifica que el sistema es completamente estable. Se recomienda realizar la demostración del proyecto utilizando la interfaz visual de Flask, ya que evidencia la mitigación del incidente INC-002 y expone una robusta gestión de estado (usabilidad) superior a las líneas de comandos tradicionales.

7.6 Aprobación:

- **Probador / Líder de QA:** Alan Oliver López Robles.
- **Fecha:** 29 de mayo de 2026.

